PostgreSQL User Group Belgium

contentgrid BY AMEXIO

Scaling Semantic Models

Bespoke PostgreSQL Schemas at SaaS Scale

Thijs Lemmens · March 2026

About Xenit & Amexio Group

Alfresco ECM Service Provider

Long-time Alfresco implementation partner — now building and offering ContentGrid as our own cloud-native ECM product.

European Digital Solutions Group

Pan-European group specialising in ECM, digital transformation, and intelligent information management.

Xenit was acquired by Amexio Group in April 2025 — bringing ContentGrid into a broader portfolio of ECM expertise across Europe.

Agenda

1. The problem with EAV / generic data models
2. Model-driven schema generation
3. Runtime platform & row-level security
4. Conclusions

contentgrid
BY AMEXIO

The Problem

Why generic EAV tables fall short

contentgrid
BY AMEXIO

The EAV Trap

How most CMS platforms store content

Generic EAV schema

```
CREATE TABLE alf_node (
  id      bigint PRIMARY KEY,
  type_qname_id bigint REFERENCES alf_qname
);
CREATE TABLE alf_qname (
  id        bigint PRIMARY KEY,
  local_name varchar(200)
);
CREATE TABLE alf_node_properties (
  node_id       bigint REFERENCES alf_node,
  qname_id      bigint REFERENCES alf_qname,
  string_value  varchar(1024),
  boolean_value bit,
  int_value     int8,
  double_value  float8,
  ...
);
```

What you actually want

```
CREATE TABLE articles (
  id            uuid PRIMARY KEY,
  title         text        NOT NULL,
  published_at timestamptz,
  word_count    integer,
  author_id     uuid REFERENCES authors
);
```

contentgrid BY AMEXIO

One Document in EAV

Storing a single article takes rows across 3 tables

alf_node — 1 row

ID	TYPE_QNAME_ID
42	7

alf_qname — excerpt

ID	LOCAL_NAME
1	title
2	published_at
3	word_count
4	author_id
7	article

alf_node_properties — 4 rows (the actual values)

NODE_ID	QNAME_ID	STRING_VALUE	BOOLEAN_VALUE	INT_VALUE	DOUBLE_VALUE
42	1	**Getting Started with PostgreSQL**	*NULL*	*NULL*	*NULL*
42	2	**2024-03-15T09:00:00Z**	*NULL*	*NULL*	*NULL*
42	3	*NULL*	*NULL*	**1842**	*NULL*
42	4	**usr_abc123**	*NULL*	*NULL*	*NULL*

contentgrid BY AMEXIO

Same Document, Bespoke Table

One row. Every column typed and named.

articles — 1 row

id uuid	title text	published_at timestamptz	word_count integer	author_id uuid
42	Getting Started with PostgreSQL	2024-03-15 09:00:00+00	1842	usr_abc123

1 row · 0 NULLs · every value in the right type

contentgrid BY AMEXIO

EAV: The Hidden Costs (1/2)

Data integrity

Concern	EAV	Native schema
Type safety	One column per type; wrong column = silent data loss	Enforced by the column type
Constraints	`NOT NULL`, `UNIQUE` cannot be expressed	Declarative, DB-enforced
Referential integrity	Application-enforced	Foreign keys

contentgrid BY AMEXIO

EAV: The Hidden Costs (2/2)

Performance & tooling

Concern	EAV	Native schema
Query complexity	Reconstruct one node via dozens of `JOIN`s	Plain `SELECT`
Query planning	Value-column statistics mix all property types — per-attribute selectivity is invisible to the planner	Accurate per-column statistics
Index efficiency	Index on `(qname_id, string_value)` covers all attributes	Targeted per-column indexes
Tooling compatibility	Breaks ORMs, analytics tools	Works out of the box
Search performance	Sync to Solr/Elasticsearch — operational heavy, denormalizes relations, eventual consistency	Native full-text search, joins, transactional consistency

> *EAV trades correctness and performance for schema flexibility — but you can have both.*

contentgrid BY AMEXIO

How Bad Is It? A Real Migration

The scenario

- **250 million** documents
- **60 attributes** each
- EAV model on Oracle

How large is the Oracle database?

A	< 500 GB
B	500 GB – 2 TB
C	2 TB – 5 TB
D	> 5 TB

contentgrid BY AMEXIO

The EAV Oracle database: 6 TB

- 15 billion property rows, each with 6+ typed value columns
- Indexes on `(qname_id, *_value)` for every attribute type

> *Now — same data, native PostgreSQL schema. Your guess?*

contentgrid
BY AMEXIO

Same Data, Native PostgreSQL Schema

What changes

- One row per document
- Typed columns — no attribute explosion
- No Oracle overhead

How large is the PostgreSQL database?

A	Still ~6 TB
B	1 TB – 3 TB
C	250 GB – 1 TB
D	< 250 GB

contentgrid BY AMEXIO

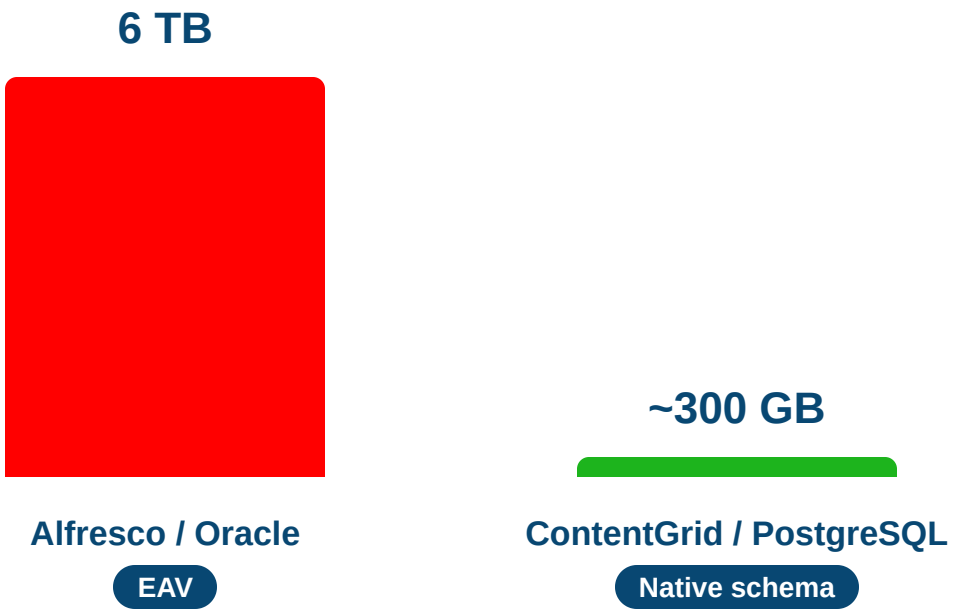
The ContentGrid database: ~300 GB

- 250M rows of typed data
- Targeted indexes
- No attribute row explosion

20× smaller — by fixing the data model

contentgrid
BY AMEXIO

Storage at a Glance



contentgrid
BY AMEXIO

Our Approach

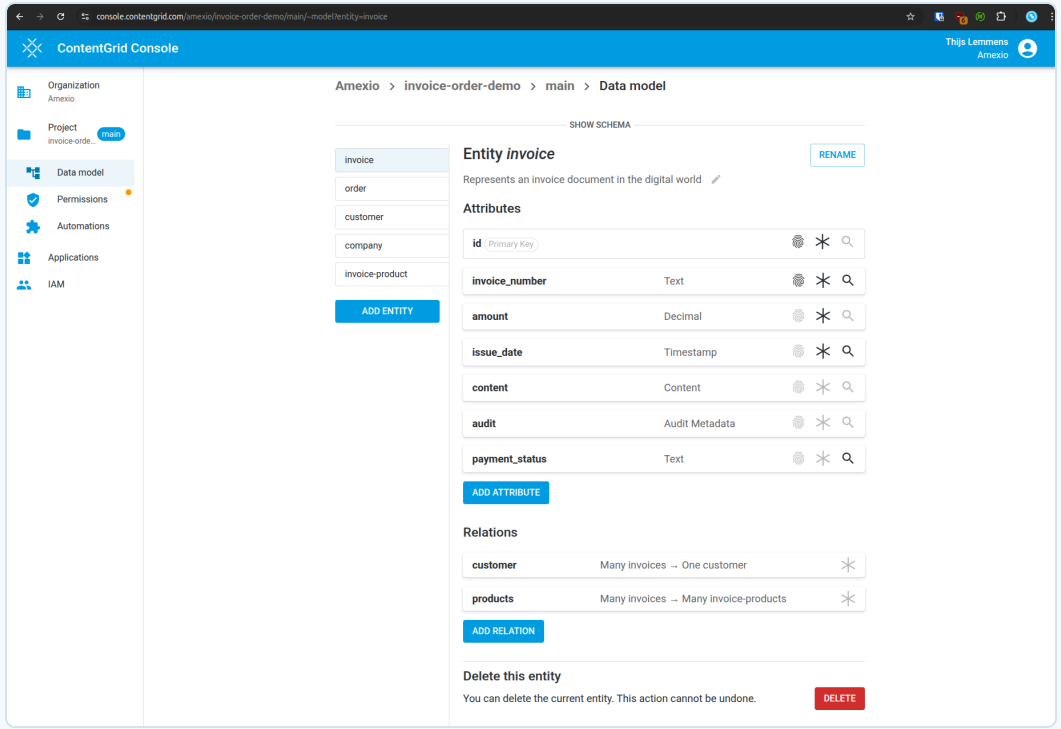
Generating native schemas from semantic models

contentgrid
BY AMEXIO

Two Platforms

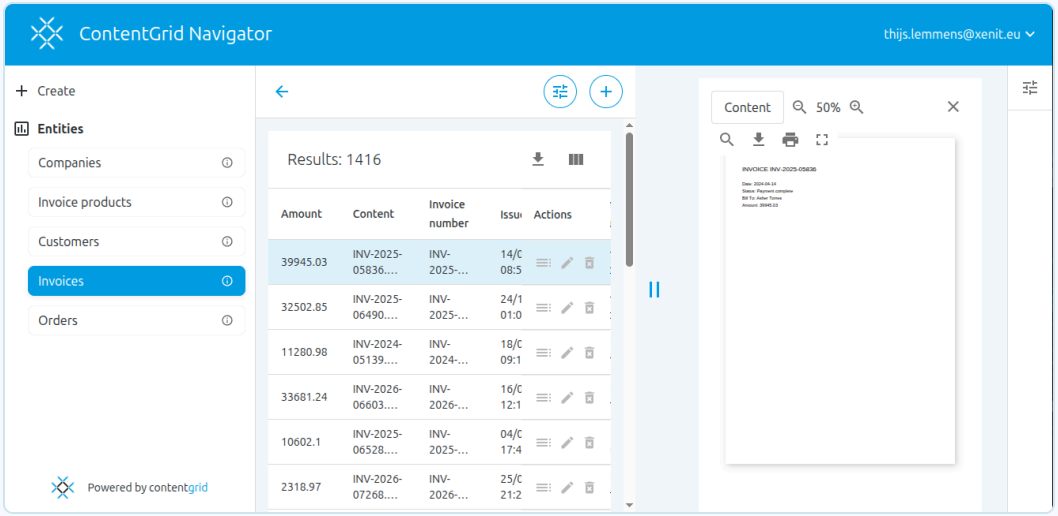
One to define the model. One to run it.

MANAGEMENT PLATFORM



ContentGrid Console — define entities, attributes, permissions

RUNTIME PLATFORM



ContentGrid Navigator — end-user app, driven by the model

contentgrid BY AMEXIO

The Management Platform

Architect

Source of truth for the domain model — entities, attributes, relations

→

Scribe

Compiles model changes into:

- JSON model
- Flyway SQL migrations
- OPA policies

→

Captain

- Provisions database
- manages credentials
- deployment

Every model change → versioned, reviewed SQL migration · Schema always reflects the model — no manual DDL

contentgrid BY AMEXIO

Model-Driven Schema Generation

User-defined semantic model

```
{
  "name": "article",
  "table": "article",
  "attributes": [
    {
      "type": "simple",
      "name": "title",
      "dataType": "text",
      "columnName": "title",
      "constraints": [{ "type": "required" }]
    },
    {
      "type": "simple",
      "name": "publishedAt",
      "dataType": "datetime",
      "columnName": "published_at"
    },
    {
      "type": "simple",
      "name": "wordCount",
      "dataType": "long",
      "columnName": "word_count"
    }
  ],
  ...
}
```

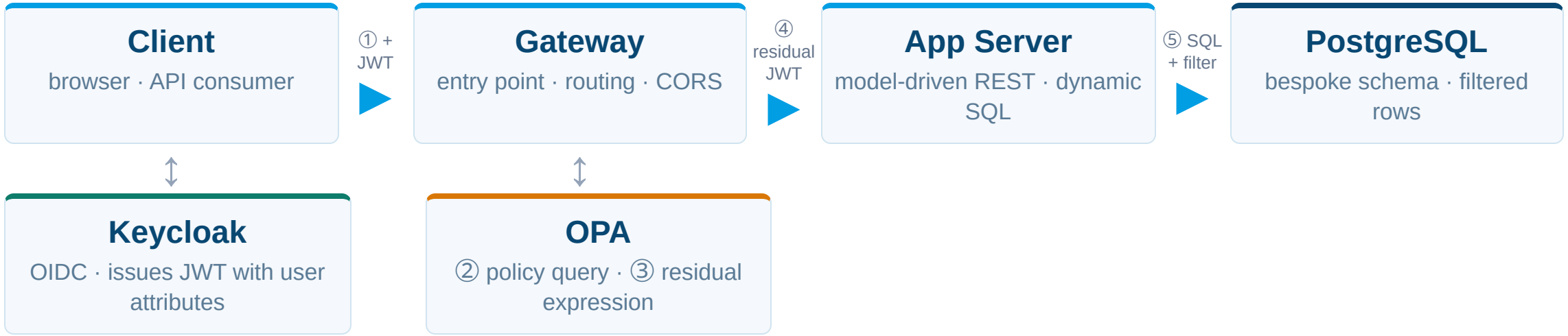
Generated PostgreSQL DDL

```
CREATE TABLE article (
  id           uuid PRIMARY KEY
                 DEFAULT gen_random_uuid(),
  title        text NOT NULL,
  published_at timestamptz,
  word_count   bigint
);
```

contentgrid BY AMEXIO

The Runtime Platform

How a request flows from client to database



Gateway encodes OPA's residual as a JWT claim — App Server applies it as a SQL `WHERE` filter

contentgrid BY AMEXIO

Partial Evaluation: The Key Insight

Naïve approach

```
for each row in invoices:        ← full table scan
  if OPA.check(row, user):       ← N network calls
    return row
```

N rows fetched, N OPA calls, unauthorized data in memory

Partial evaluation

```
OPA.partial_eval(policy, user)  ← 1 call, no entity data
    → residual: dept='sales'
      OR status='published'

SELECT * FROM invoices
WHERE dept='sales'
  OR status='published'          ← filtered at the DB
```

1 OPA call, unauthorized rows never leave PostgreSQL

The security boundary is enforced inside the database — not in application memory

contentgrid BY AMEXIO

From Policy Rule to SQL Filter

① Policy condition

```
entity.department == user.department
OR entity.status == "published"
```

② OPA partial eval

```
OPA.partial_eval(policy, user)
  → dept == user.dept  OR  status == "published"
```

③ SQL WHERE (JOOQ)

```
SELECT * FROM invoices
WHERE department = 'sales' OR status = 'published'
```

Defined in Architect UI → compiled to Rego by Scribe → evaluated by OPA (1 call, result in Gateway JWT) → App Server builds JOOQ predicate

Unauthorized rows never leave the database — the filter runs inside PostgreSQL

contentgrid BY AMEXIO

Conclusions

- EAV trades correctness & performance for flexibility — you can have both
- Native schemas: 20× smaller, full SQL tooling, real constraints
- ContentGrid generates & migrates schemas from a semantic model
- Security boundary enforced inside PostgreSQL via partial evaluation

contentgrid
BY AMEXIO

Questions?

contentgrid.com

contentgrid
BY AMEXIO

Thank You

Thijs Lemmens

thijs.lemmens@xenit.eu